Access-Induced Semantic Divergence

Access-Induced Semantic Divergence in Generated Program Transformations

Abstract

Access-induced semantic divergence occurs when an operation that appears observational updates latent state and changes a later externally visible result. We study this phenomenon through an observation-sensitive deterministic-state (OSDS) model, proof obligations for a straight-line core, real-package witnesses, and coding-agent transformations. The real-code study contains 20 confirmed divergences across 12 unmodified PyPI packages and 9 caller-level branch flips. A frozen coding-agent benchmark built from those witnesses evaluates behavior preservation under ordinary tests and OSDS-aware metamorphic checks. In the primary benchmark, gpt-5.6-sol produced 0 verified OSDS divergences in 26 tasks, gpt-5.6-terra produced 2, and gpt-5.6-luna produced 3. All five verified failures passed ordinary tests. A causal-control replay reproduced all five failures and removed all five when the identified access-induced mechanism was neutralized. A prospectively frozen 7-witness expansion adds 14 tasks and 42 Sol/Terra/Luna generations; all 42 preserved behavior. The results support a bounded claim: generated transformations can silently violate access-sensitive semantics, and OSDS-aware checks plus mechanism controls make such failures reproducible and falsifiable.

1. Introduction

Many program transformations treat reads, logging, inspection, representation, and caching as harmless local changes. That assumption fails when the accessed object carries latent state. A read may advance a cursor, update a cache statistic, materialize a stream, change a handler level, or populate an identity map. The later program can then observe a different value even when the added operation did not appear to change the explicit return value at the point where it was inserted.

This paper studies this failure mode as access-induced semantic divergence. The term does not mean that every observation is unsafe. It identifies a semantic pattern: an operation with an observational surface changes latent state that a later read can expose. The target setting is software verification for generated transformations, where ordinary tests can pass while an access-sensitive metamorphic check fails.

The contribution is a combined formal and empirical account. We give an OSDS model for read and observation transitions, state proof obligations for deterministic and zero-divergence cases, reproduce real-package divergences, and evaluate coding-agent transformations against frozen tasks derived from those witnesses. The coding-agent study is not a prevalence estimate. It is a controlled test of whether generated behavior-preserving edits respect access-sensitive semantics.

2. OSDS Model

An OSDS value has a stable component and latent access state. In the proof core a semantic value is (b, a, d), where b is stable, a records read count, and d records latent drift. A read transition exposes f(b, a, d) and updates latent state. An observation transition exposes no additive value but updates latent drift through a deterministic function g. A program body folds those transitions over an operation list and applies a deterministic cap.

This model separates exposed values from latent effects. A logging call, representation call, cache lookup, or stream inspection can be modeled as an observation when it contributes no intended body value but may update d. Divergence appears when two orderings contain the same operations but reach a later read with different latent state.

The proof appendix establishes four bounded facts for the studied straight-line template: fixed-order determinism, zero divergence for identity observations, zero divergence for access-insensitive reads, and preservation of body-level divergence under a nonzero-slope linear cap. The proofs deliberately avoid claims about arbitrary Python programs, analyzer soundness, or production prevalence.

3. Real-Code Evidence

The real-code oracle study instantiates the OSDS transition structure on unmodified package operations. From 60 selected candidates, 39 executable harnesses were constructed. The metamorphic oracle found 20 confirmed divergences across 12 packages: httpcore, PyYAML, pytest, markdown, more-itertools, docutils, beautifulsoup4, boltons, cerberus, dnspython, h11, and anyio.

The divergences include stream materialization in httpcore.Response, identity-cache reuse in PyYAML, handler-level mutation in pytest, reference-registry mutation in markdown, cursor advance in more-itertools and dnspython, destructive tree extraction in beautifulsoup4, cache recency and statistics in boltons, validation-error population in cerberus, and buffer consumption in h11.

Nine confirmed divergences were lifted into caller-level branch flips. Each wrapper changed both a branch label and a downstream consequence, such as cache versus stream handling, alert emission versus suppression, request acceptance versus rejection, and recomputation versus cache serving. Nineteen negative controls removed the divergence under fresh-object, reset-between, or pure-observation interventions.

4. Coding-Agent Benchmark

The primary coding-agent benchmark was frozen before model execution. It contains 13 base tasks and 26 normal/warned prompt variants from 9 packages and 9 unique witnesses. Each task asks for a small behavior-preserving Python transformation. The model sees only the code and the editing instruction. It does not see oracle labels, witness labels, benchmark explanations, or prior responses.

Evaluation uses exact-response replay. The runner extracts Python, executes ordinary smoke tests, then compares the baseline and candidate under an OSDS-aware metamorphic oracle. Failures are manually classified as verified semantic divergence, ordinary programming bug, invalid patch, environment failure, oracle issue, or unclear. A verified OSDS divergence requires an executable transformation, real behavioral divergence, OSDS oracle detection, and manual confirmation that the mechanism is access-induced.

5. Primary Model Results

The primary benchmark evaluated three Codex task-model configurations at low reasoning effort with temperature and seed unavailable through the task interface. gpt-5.6-sol produced 26 executable candidates, with 23 behavior-preserving candidates, 3 ordinary programming bugs, and 0 verified OSDS divergences. gpt-5.6-terra produced 24 executable candidates, with 18 behavior-preserving candidates, 4 ordinary bugs, 2 invalid patches, and 2 verified OSDS divergences. gpt-5.6-luna produced 24 executable candidates, with 17 behavior-preserving candidates, 4 ordinary bugs, 2 invalid patches, and 3 verified OSDS divergences.

All five verified OSDS failures were silent under ordinary tests. Terra failed on pytest_catching_logs__instrumentation__normal and pytest_catching_logs__instrumentation__warned. Luna failed on those two pytest rows and on pyyaml_representer__caching_materialization__normal. Each failure appeared in a hidden-observation case. The expected-access-sensitive calibration rows produced ordinary bugs or invalid patches, with no verified OSDS divergence.

Self-assessment was conservative in the primary study. Across Sol, Terra, and Luna, there were zero false YES preservation claims on verified OSDS divergences. The self-assessment results are secondary because the primary evidence is behavioral replay.

6. Causal Controls

The causal-control experiment replays the exact generated candidate files for the five verified failures. The candidate source remains byte-identical. The intervention changes only the witness or environment mechanism that caused the access-induced divergence.

For pytest, the control isolates diagnostic logging from the captured handler hierarchy so that the logging-shaped observation no longer mutates the handler state read later by the program. For PyYAML, the control clears the relevant identity/access cache state after the representer observation while retaining the generated caching transformation. Under the original witness environment, all five failures reproduce: ordinary tests pass and OSDS checks fail. Under the mechanism-neutralizing control, all five OSDS divergences disappear. The causal status for all five rows is mechanism_neutralized_divergence_disappeared.

This result is important because it narrows the explanation. The failures are not merely arbitrary generated-code defects. They are reproduced by the exact generated transformations and removed by targeted neutralization of the access-induced mechanism.

7. Prospective Expansion

After the causal-control phase, the unused confirmed real-code witness pool was audited prospectively. Eleven confirmed witnesses were unused by the primary benchmark. Seven met the eligibility criteria: exact package reconstruction, baseline execution, oracle reproduction, caller/control availability where applicable, and automated task execution. The expansion was frozen at commit 0f7dea5ca3cfc62e040026a8c780f1127526b0dc before any model execution.

The frozen expansion contains 7 new base tasks and 14 normal/warned variants from 3 packages and 7 witnesses: boltons LRI statistics, boltons MultiFileReader, h11 ReceiveBuffer, boltons SpooledStringIO, boltons SpooledBytesIO, dnspython Tokenizer concatenation, and a second boltons LRU pair witness. All expansion rows are expected-access-sensitive calibration cases.

Sol, Terra, and Luna were run on the exact frozen expansion prompts as fresh projectless Codex tasks. Self-assessment was skipped in the cut-scope completion. The three replays produced 42 executable candidates. All 42 passed ordinary tests and OSDS-aware checks. No expansion row was classified as an ordinary bug, invalid patch, verified OSDS divergence, silent divergence, environment failure, oracle issue, or unclear. Normal prompts preserved behavior in 21/21 rows; warned prompts preserved behavior in 21/21 rows.

8. Model Differential Findings

The primary benchmark contains the model-differential evidence. Sol preserved all verified-OSDS rows where Terra or Luna diverged. Terra diverged on two pytest instrumentation rows. Luna diverged on those same two rows and on one PyYAML caching row. The prospective expansion contains no model-differential OSDS failures: all 14 tasks were preserved by Sol, Terra, and Luna.

The expansion result should be read as a boundary, not a contradiction. The added witnesses were expected-access-sensitive cases where the access-sensitive operation is visible in the code shape. The primary failures occurred in hidden-observation cases where a generated logging or caching edit looked locally harmless and ordinary tests did not expose the later state-dependent effect.

9. Verification Implications

Ordinary smoke tests are insufficient for this class of transformations. The five primary OSDS failures passed ordinary tests because the visible single-order behavior remained plausible. The OSDS oracle detected divergence by comparing orderings that differ only in the placement of an observation-shaped access on an equivalent object.

The causal controls provide a practical validation pattern. A semantic-preservation claim is stronger when a failure reproduces under the original witness and disappears under a mechanism-neutralizing intervention that leaves the generated candidate unchanged. This pattern helps distinguish access-induced semantic divergence from unrelated programming bugs.

10. Limitations

The coding-agent benchmark is small and correlated by witness and package. Counts are benchmark outcomes, not prevalence estimates. Codex task-model runs are real-model results, but they were collected through Codex projectless tasks rather than the OpenAI API. Temperature and seed were not exposed by the task interface. The prospective expansion adds seven witnesses but all are expected-access-sensitive; it does not add new hidden-observation witnesses.

Manual review remains part of the verified OSDS classification. The formal core covers a straight-line deterministic template and does not prove analyzer soundness or general Python semantics. The real-code study uses selected candidates and constructed harnesses, so it supports existence, mechanism, and reproducibility rather than ecosystem prevalence.

11. Conclusion

Access-induced semantic divergence is a concrete risk for generated behavior-preserving transformations. In real package code, observation-shaped operations can change later externally visible behavior through latent state. In the primary coding-agent benchmark, five verified silent OSDS failures were found across Terra and Luna, all missed by ordinary tests. Exact candidate replay plus targeted causal controls reproduced those failures and removed all five when the identified mechanism was neutralized. The prospective expansion found no new failures across Sol, Terra, and Luna, which constrains the claim and strengthens the paper: the strongest evidence is not a broad failure rate, but a reproducible semantic failure mode with real-code witnesses, model-generated instances, and mechanism-level controls.